

NER Pipeline on the Spark Cluster

The complete process from start to stop: how the models get onto the cluster (HDFS or the master's file system), how a job is submitted and distributed, what happens to every document, and how the cluster is shut down.

Scope: the ner_translate pipeline (OCR -> language detection -> NLLB-200 translation -> GLiNER entity extraction) running on this platform's Spark cluster, with models held in an air-gapped HDFS cluster or on the master node's file system. Everything described was executed in the simulation of section 7 (deploy/docker-compose.airgap_sim.yml, deploy/airgap_sim_tests.ps1). Operational commands are in the companion CLUSTER_RUNBOOK_MODES_AND_TESTS.pdf.

Contents

1. The big picture
2. The components and their roles
3. Where the models live - the model store
4. The process, phase by phase (0 build ... 7 stop)
5. What happens to one document
6. When things go wrong
7. Measured in the simulation
8. Glossary

1. The big picture

24. NER on the cluster - complete lifecycle from build to shutdown

Left to right, top to bottom in time; each phase lists what runs and what state it leaves behind

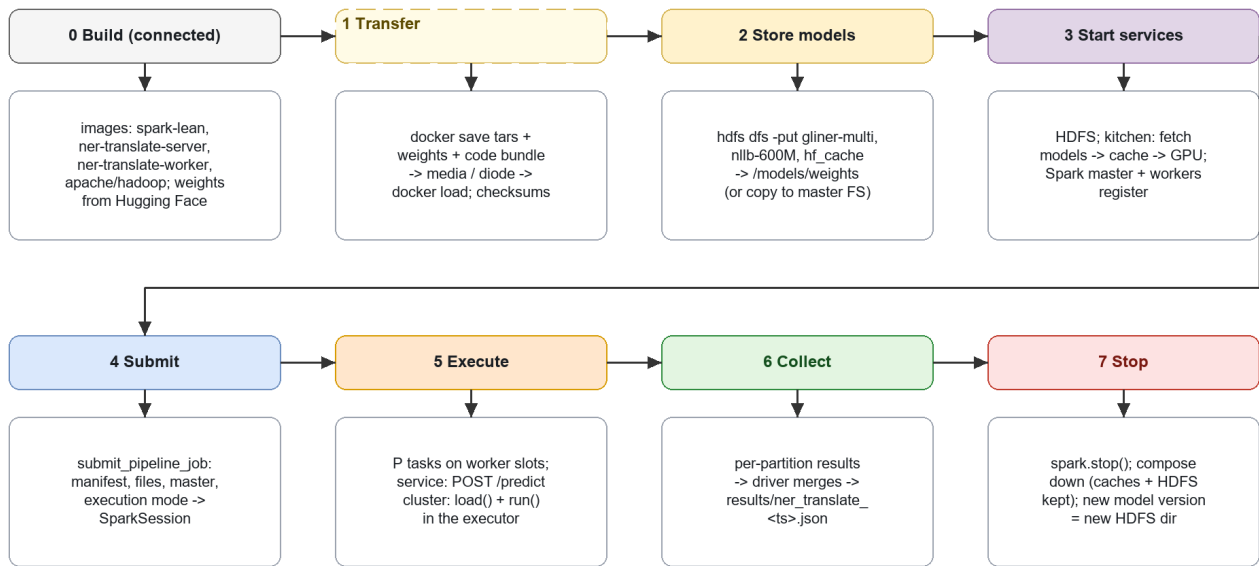


Figure 1 - the eight phases in time order

A user points `submit_pipeline_job.py` at a folder of documents. Spark splits the list of files into partitions and hands each partition to an executor on a worker node. Each partition's documents are processed by the NER pipeline - either inside a central GPU model server (the **kitchen**, service mode) or inside the executor itself (cluster mode). The per-document results flow back to the driver and are written to one JSON file. The models themselves never live in the images: they are stored once in the cluster's HDFS (or on the master's file system) and every node that needs them fetches them into its own local cache the first time.

phase	what happens	state it leaves behind
0 Build	images and model weights prepared on a connected machine	4 images, 3 model folders (4.6 GB)
1 Transfer	images saved as tar files, copied with the weights across the air gap, loaded	images on every node
2 Store models	gliner-multi, nllb-200-distilled-600M and hf_cache put into HDFS /models/weights (or copied to the master's shared file system)	one authoritative copy
3 Start services	HDFS, then the model server (fetch -> cache -> GPU), then Spark master and workers	warm, idle cluster
4 Submit	driver reads the manifest, lists files, picks master and execution mode	SparkSession
5 Execute	tasks run on worker slots; documents are extracted, translated, tagged	per-partition results
6 Collect	results merged on the driver, written to results/ner_translate_<time>.json	result file
7 Stop	SparkSession stopped; services stopped when no more jobs are expected	HDFS + node caches kept

2. The components and their roles

component	role	runs where (simulation)	image
HDFS namenode	keeps the file-system tree (which blocks make up which model file); answers WebHDFS requests and redirects reads to a datanode	sim-hdfs-namenode (master storage)	apache/hadoop:3.4.1
HDFS datanode	stores the 128 MB blocks of the model files and streams them	sim-hdfs-datanode	apache/hadoop:3.4.1
Model store (models/model_store.py)	turns hdfs:// or file:// model locations into a local folder; downloads once per node, caches, publishes atomically	inside the kitchen and inside executors	(code, stdlib only)
Model server - 'kitchen' (serve.py)	loads GLiNER + NLLB once on the GPU, serves POST /predict {paths}	sim-ner-translate-server	ner-translate-server
Spark master	tracks workers, grants executors to the job	sim-spark-master / sim-ner-cluster-master	spark-lean / ner-translate-worker

component	role	runs where (simulation)	image
Spark driver (submit_pipeline_job.py)	builds the job, collects results, writes the JSON	in the master container	same as master
Spark worker + executor	runs tasks: service mode - posts file paths to the kitchen; cluster mode - loads models and runs NER itself	sim-spark-worker / sim-ner-cluster-worker	spark-lean / ner-translate-worker
Shared data volume	documents to process; the same path (/app/data/...) is valid on every node	../data mounted everywhere	-

Why two execution modes? **Service mode** keeps the heavy dependencies (torch, CUDA, transformers, tesseract) and the GPU in one server; Spark nodes stay lean and the models are loaded once. **Cluster mode** needs every executor to carry the full dependency set and load the models itself (once per task), but needs no separate server.

3. Where the models live - the model store

21. Air-gapped cluster simulation: models in HDFS, nodes fetch into a local cache

deploy/docker-compose.airgap_sim.yml - profiles hdfs / service / cluster; MODEL_STORE_URI selects where models come from

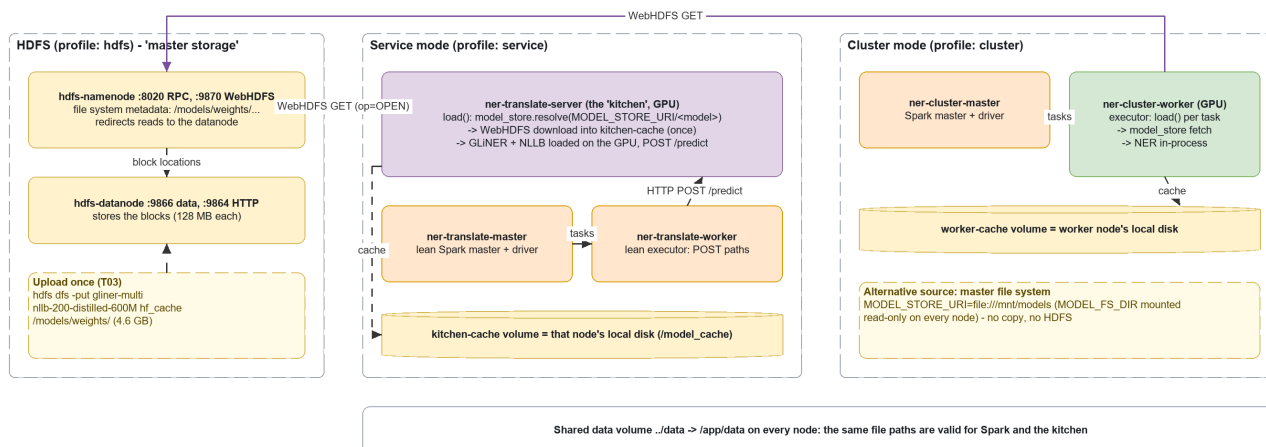


Figure 2 - models in HDFS; the kitchen (service mode) and the executor (cluster mode) each keep a node-local cache

The pipeline's models are three folders: **gliner-multi** (GLiNER weights, 2.2 GB), **nllb-200-distilled-600M** (translation model, 2.4 GB) and **hf_cache** (4 MB: config and tokenizer of GLiNER's backbone, microsoft/mdeberta-v3-base, in Hugging Face cache layout - GLiNER looks its encoder up by that hub name, so without it GLiNER cannot load offline). A single setting, `MODEL_STORE_URI`, tells every node where the three folders are:

MODEL_STORE_URI	meaning	what a node does
hdfs://hdfs-namenode:8020/models/weights	models in the cluster's HDFS (air-gapped master storage)	downloads each model folder once via WebHDFS into MODEL_CACHE_DIR, then loads from there
file:///mnt/models	models on the master's file system, shared read-only with the nodes (NFS or similar)	loads straight from the shared path - no copy
(unset)	models next to the code in models/weights (original behaviour)	loads from the repository folder

22. How a node gets a model from HDFS (models/model_store.py)

WebHDFS = HDFS's HTTP API; only the Python standard library is needed on the node

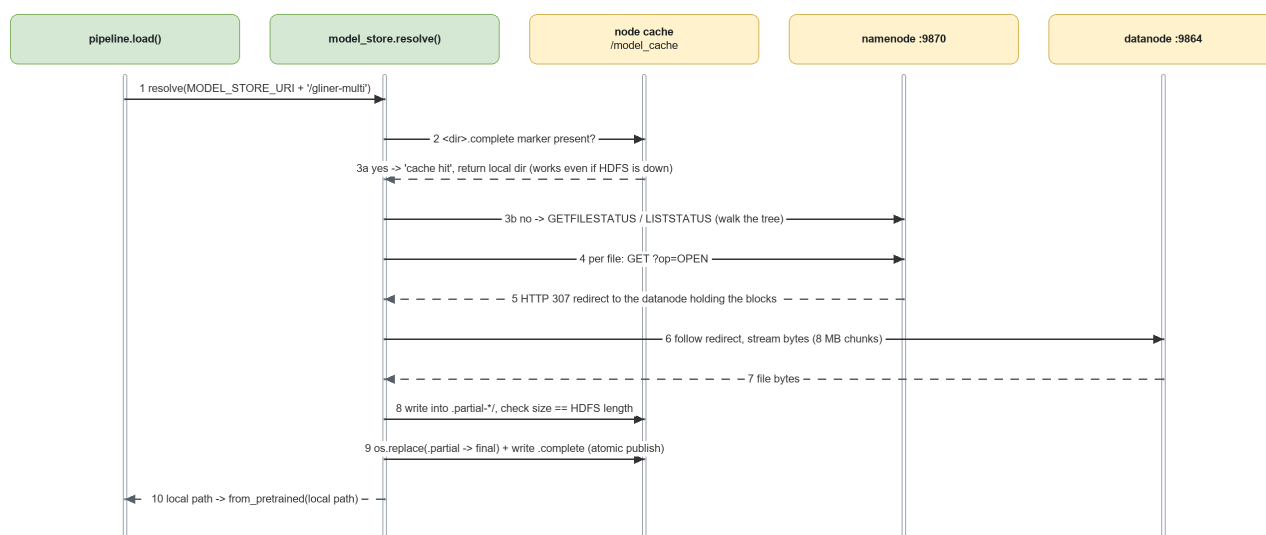


Figure 3 - one model fetch, step by step

- **WebHDFS** is HDFS's built-in HTTP interface: the namenode answers GET /webhdfs/v1/<path>?op=OPEN with a redirect to the datanode that holds the blocks, and the datanode streams the bytes. The node needs nothing but Python's standard library - no Hadoop client, JVM or extra packages.
- **Once per node.** A finished download leaves <folder>.complete; later loads on that node (every task in cluster mode, every restart of the kitchen) are cache hits and do not touch HDFS.
- **Atomic.** Files are written into a .partial-* folder, sizes are checked against HDFS, and the folder is renamed into place in one step - two Python workers on the same node can never read a half-written model.
- **Offline tolerant.** With a warm cache a node starts even while HDFS is down (test T09).
- **Versioning.** Put a new model version into a new HDFS folder (e.g. /models/weights/v2/...) and point MODEL_STORE_URI at it; old and new caches coexist.

4. The process, phase by phase

Phase 0 - Build (on a connected machine)

Build the four images, pull Hadoop, and download the three model folders from Hugging Face once:

```

docker build --target final -t multi-model-inference:latest -f deploy/Dockerfile .
docker build --target lean -t spark-lean:latest -f deploy/Dockerfile .
docker build -t ner-translate-worker:latest -f deploy/Dockerfile.ner_translate .
docker build -t ner-translate-server:latest -f deploy/Dockerfile.ner_translate_server .
docker pull apache/hadoop:3.4.1

python -c "from gliner import GLiNER; GLiNER.from_pretrained('urchade/gliner_multi-v2.1').save_pretrained('models/weights/gliner-multi')"
python -c "from transformers import AutoModelForSeq2SeqLM, AutoTokenizer as T; m='facebook/nllb-200-distilled-600M';
T.from_pretrained(m).save_pretrained('models/weights/nllb-200-distilled-600M');
AutoModelForSeq2SeqLM.from_pretrained(m).save_pretrained('models/weights/nllb-200-distilled-600M')"
# GLiNER's backbone, config + tokenizer only, in Hugging Face cache layout:
python -c "from huggingface_hub import snapshot_download as s; s('microsoft/mdeberta-v3-base',
cache_dir='models/weights/hf_cache', allow_patterns=['*.json', 'spm.model'])"
    
```

Phase 1 - Transfer across the air gap

docker save each image to a tar file, pack the three model folders, record SHA-256 checksums, move them on approved media, verify the checksums on the other side and docker load the images on every node. Code changes afterwards travel as a small bundle (the repository without weights).

Phase 2 - Store the models in HDFS (or on the master)

```
hdfs dfs -mkdir -p /models/weights
hdfs dfs -put gliner-multi nllb-200-distilled-600M hf_cache /models/weights/
hdfs dfs -du -s -h /models/weights/*           # sizes must match the source folders
```

Alternative: copy the three folders to a directory on the master that every node mounts read-only (e.g. /mnt/models) and use MODEL_STORE_URI=file:///mnt/models.

Phase 3 - Start the services

1. **HDFS** first (namenode, then datanodes). Healthy when WebHDFS answers and `hdfs dfsadmin -report` shows live datanodes.
2. **Model server (service mode)**. On startup `serve.py` calls `pipeline.load()`: the model store resolves the three folders (download on first start, cache hit afterwards), the `hf_cache` entries are copied into the process's Hugging Face cache, then GLiNER is moved to the GPU in fp16, NLLB-200 (600 M parameters) in fp32, and the language identifier (`py3langid`) is ready. `/health` switches from 'loading' to 'ok'.
3. **Spark master, then workers**. Each worker registers its cores and memory with the master (visible on the master UI). In cluster mode the worker node also needs the GPU and the full NER dependency set.
4. Nothing is processed yet - the cluster is warm and idle.

Phase 4 - Submit a job

```
python submit_pipeline_job.py --pipeline ner_translate --input data/ner_samples \
    --partitions 2 --execution-mode service --master spark://ner-translate-master:7077
```

1. Look up `ner_translate` in `models/pipelines/manifest.json` (module, `service_url`, `master_url`).
2. Collect the input files (a folder is walked recursively; globs are expanded).
3. Choose the Spark master: `--master`, else `SPARK_MASTER_URL`, else the manifest's `master_url` if it resolves, else `local[4]`.
4. Choose the execution mode: **service** (HTTP to `service_url`), **cluster** (import the pipeline and run it in the executors) or **auto** (service if its host resolves).
5. Create the `SparkSession` (driver and executor memory from the command line).

Phase 5 - Execute

18. Request -> response: `submit_pipeline_job.py` (cluster vs service execution)

Same partitioning; the difference is where `load()/run()` executes

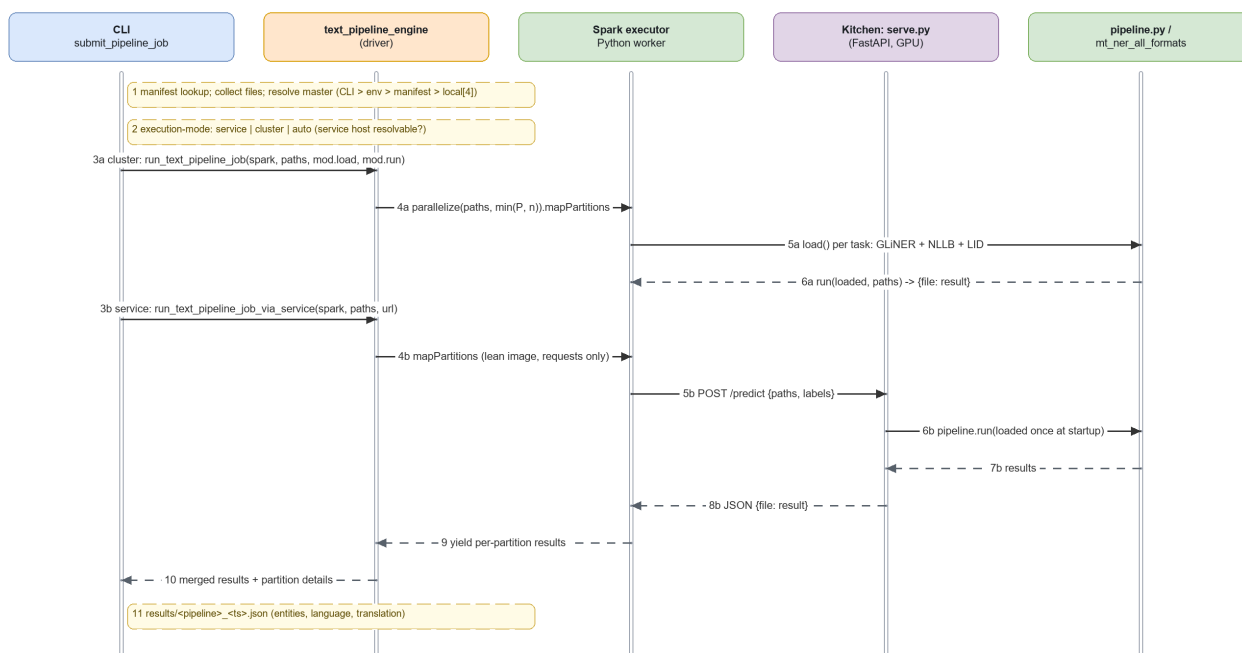


Figure 4 - the same job in cluster mode (a) and service mode (b)

The list of file paths is split into $\min(\text{partitions}, \text{files})$ partitions; each becomes one task that runs on a free worker slot. **Service mode:** the task sends its paths in one `POST /predict` to the kitchen and waits for the JSON reply - the executor never touches a model. **Cluster mode:** the task calls `load()` (model store fetch or cache hit, then GLiNER + NLLB onto the executor's GPU) and `run(loaded, paths)` itself. Paths, not file contents, are sent - so every node must see the documents at the same path (the shared data volume). The driver turns every input into an absolute path first (`/app/data/ner_samples/...`): an executor's working directory is its Spark work folder, not the driver's, so relative paths would not resolve there.

Entity counts can differ by one or two between runs in different modes (e.g. `sample_text1`: 27 in service mode with 2 partitions, 28 in cluster mode with 1 partition). Documents are batched together for the GPU, the batch composition depends on the partitioning, and GLiNER runs in fp16 - a mention scoring right at the 0.35 threshold can fall either side.

Phase 6 - Collect

Each task yields `{hostname, pid, number of paths, load seconds, run seconds, results}`. The driver merges all partitions' results into one dictionary keyed by file name and writes `results/ner_translate_<timestamp>.json`:

```
# ner_translate_20260927_092255.json
"elapsed_time": 548.4056, "num_files": 6, "num_partitions": 1,
"partition_details": [{ 'hostname': 'ner-cluster-worker', 'pid': 208, 'num_paths': 6, 'load_time_sec': 401.927, '
  run_time_sec': 88.641}],
"results": { "sample_scan6.png": { "language": "en", "translated": false,
  "entities_unique": [{ 'text': '18 June 2026', 'type': 'date', 'score': 0.8101, 'start': 3, 'end': 15}, { 'text
': '09:15 UTC', 'type': 'time', 'score': 0.4233, 'start': 19, 'end': 28}, { 'text': 'Major Rakesh Verma', 'ty
pe': 'person', 'score': 0.9468, 'start': 30, 'end': 48}, { 'text': 'Indian Army', 'type': 'nationality', 'sco
re': 0.79, 'start': 53, 'end': 64}] ... } ... }
```

Phase 7 - Stop

- The driver calls `spark.stop()` - executors and their Python workers exit; the master and workers stay up for the next job.
- When no more jobs are expected: stop the Spark services, the model server and HDFS (`docker compose ... down`). HDFS data and the node caches are volumes and survive; the next start is a cache hit.
- To change a model: upload the new version to a new HDFS folder, point `MODEL_STORE_URI` at it, restart the model server (and workers in cluster mode). To force a re-download delete the node's cache folder (or its `.complete` marker).

5. What happens to one document

23. What happens to every document (models/pipelines/ner_translate/mt_ner_all_formats.py)

process_paths_batched(): extract -> detect -> route -> translate (batched per language) -> NER (batched across documents) -> merge

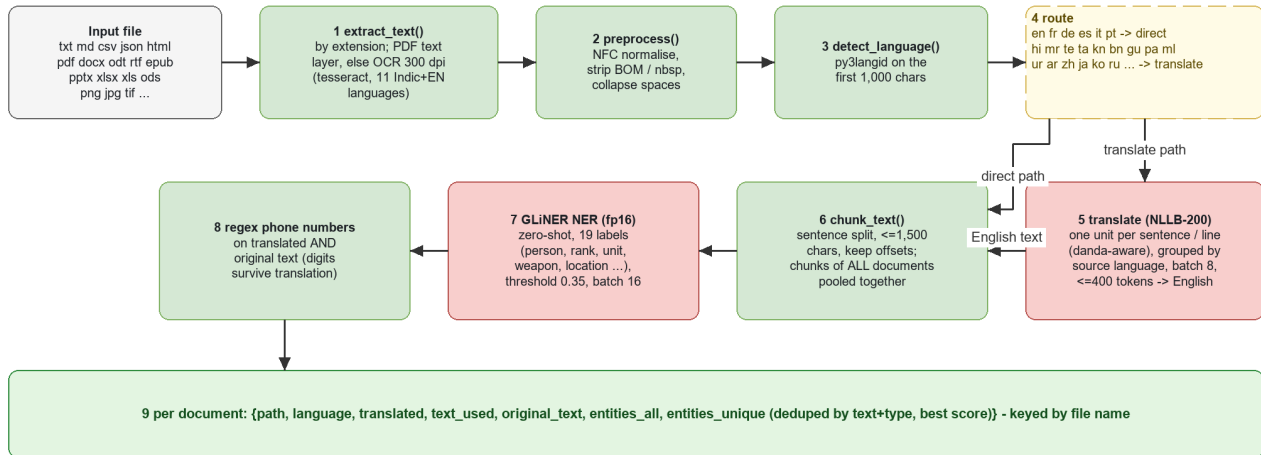


Figure 5 - the stages inside process_paths_batched()

stage	detail (from mt_ner_all_formats.py)
1 Extract	chosen by file extension: text/markdown/CSV/JSON/HTML/XML, PDF (text layer; if less than 20 characters -> OCR every page at 300 dpi), DOCX, ODT, RTF, EPUB, PPTX, XLSX/XLS/ODS (all cell text), images (OCR). OCR languages: English + Marathi, Hindi, Telugu, Tamil, Kannada, Bengali, Gujarati, Punjabi, Malayalam, Urdu (packs that are not installed are dropped automatically).
2 Preprocess	Unicode NFC, remove non-breaking spaces / BOM, collapse whitespace and blank lines.
3 Detect language	py3langid on the first 1,000 characters (offline).
4 Route	English, French, German, Spanish, Italian, Portuguese go straight to NER; languages with an NLLB code (Indic languages, Arabic, Chinese, Japanese, Korean, Russian, Persian, Turkish, Vietnamese, Thai, Indonesian) are translated first; anything else is tagged as is.
5 Translate	NLLB-200-distilled-600M, source language -> eng_Latn, one unit per sentence or line (the Devanagari danda counts as a sentence end), split further only above 400 tokens, 8 units per batch; documents in the same language are translated together. Whole paragraphs are never sent: NLLB drops text from multi-sentence input.
6 Chunk	split into sentences, re-join up to 1,500 characters, remember each chunk's offset; chunks from ALL documents are pooled so the GPU sees full batches.
7 NER	GLiNER multi v2.1, zero-shot, 19 labels (person, rank, organization, military unit, weapon system, vehicle, aircraft, vessel, equipment, facility, location, coordinates, operation name, event, date, time, nationality, communication identifier, phone number), score >= 0.35, 16 chunks per batch, fp16 on GPU.
8 Phone numbers	regex pass over the text used and, for translated documents, over the original too (digits survive translation).
9 Result	entities with type, score and character offsets; entities_unique keeps the best-scoring mention per (text, type); plus language, translated flag, text used and original text.

6. When things go wrong

situation	what happens	what to do
HDFS down, node cache warm	model server / executor starts from the cache (T09)	nothing; restart HDFS later
HDFS down, cache empty	load() fails: 'cannot reach WebHDFS at http://...' (ConnectionError)	start HDFS, or use the master file-system mode
model folder missing in HDFS	FileNotFoundError naming the path and namenode	hdfs dfs -put the folder; check MODEL_STORE_URI
interrupted download	the .partial folder is discarded; next load downloads again	nothing
kitchen still loading	/health says 'loading'; /predict answers 'models still loading'	wait for 'ok' before submitting
unreadable / unsupported file	that document gets {'error': ...}; the others continue	check the error text in the result
two inputs with the same file name	results are keyed by file name - the second overwrites the first	keep file names unique

situation	what happens	what to do
GPU or host out of memory	model load fails (CUDA OOM / container killed)	fewer concurrent tasks per GPU; larger node; service mode
hf_cache missing	GLiNER fails: 'couldn't connect to huggingface.co ... microsoft/mdeberta-v3-base'	store hf_cache next to the two models (the pipeline warns when it is absent)

7. Measured in the simulation

measurement	value	test
first fetch from HDFS: gliner-multi	2,312 MB in 118.3 s (20 MB/s)	T05
first fetch from HDFS: nllb-200-distilled-600M	2,483 MB in 130.3 s (19 MB/s)	T05
first fetch from HDFS: hf_cache	9 MB in 0.8 s (10 MB/s)	T05
model server start with HDFS fetch	PASS (461.8 s)	T05
model server restart (cache hit)	PASS (206.5 s)	T08
start with HDFS down (warm cache)	PASS (304.3 s)	T09
Spark job, service mode, 6 documents	PASS (96.6 s)	T07
master file-system mode (start + job)	PASS (359.8 s)	T10
cluster mode: executor fetch + load + NER	PASS (620.9 s)	T11

Machine: laptop, Intel i5-9300H, 8 GB RAM (6 GB given to Docker/WSL2), NVIDIA GTX 1650 4 GB, Docker Desktop on Windows 11.
On a real cluster the fetch runs at the network / HDFS rate instead of the Windows file-sharing rate.

8. Glossary

term	meaning
air gap	network with no connection to the internet; software and models arrive as files
HDFS / namenode / datanode	Hadoop distributed file system / its metadata server / its block storage servers
WebHDFS	HDFS's HTTP REST API (port 9870 on the namenode, 9864 on datanodes)
model store	models/model_store.py - resolves hdfs:// or file:// model locations to a local folder, with a per-node cache
kitchen / waiter	the model server that holds the models / the lean Spark job that sends it work
driver / executor / task / partition	job-building process / worker-side process / one partition's work / slice of the input files
GLiNER / NLLB / py3langid	zero-shot entity extractor / Meta's 200-language translator / offline language identifier